Compte Rendu – TP 1

UniAccounts – Design Patterns

إعداد الطالبة: عياشي هاجر

الفوج: الأول

لغة البرمجة: Java

السنة الجامعية: 2026–2027

UML Class Diagram

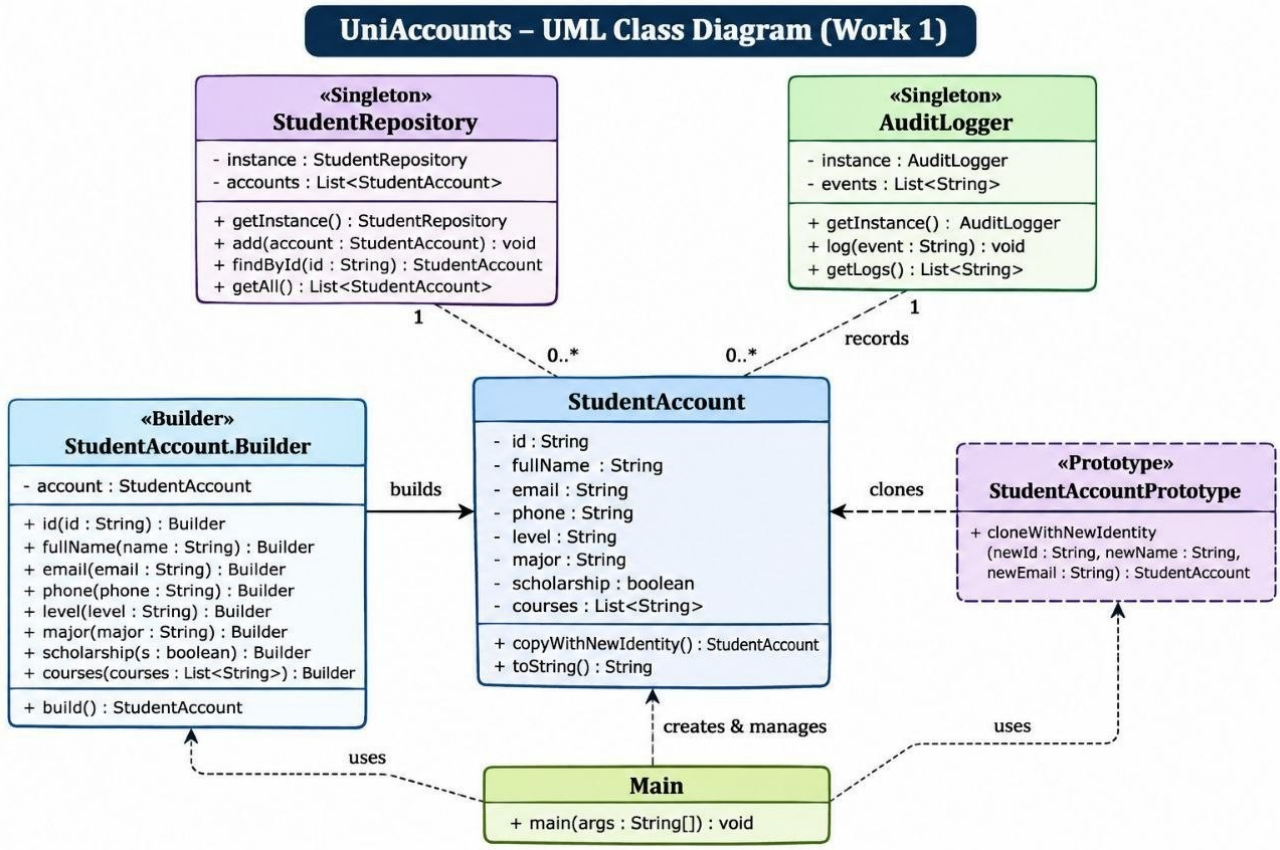
1. مخطط الأصناف والعلاقات

يوضح مخطط UML البنية العامة لمشروع UniAccounts، ويبين الأصناف المستعملة والعلاقات بينها، بالإضافة إلى كيفية تطبيق أنماط التصميم الثلاثة: Builder وSingleton وPrototype.

يتكون النظام من الأصناف الأساسية التالية:

- StudentAccount: يمثل حساب الطالب ويحتوي على معلوماته الشخصية والأكاديمية.
- StudentAccount.Builder: الصنف الداخلي المسؤول عن إنشاء حساب الطالب بطريقة تدريجية.
- StudentRepository: مسؤول عن تخزين وإدارة حسابات الطلبة، ويطبق Singleton.
- AuditLogger: مسؤول عن تسجيل العمليات التي تتم داخل النظام، ويطبق Singleton.
- Main: الصنف الرئيسي الذي يستعمل الأنماط ويختبر عملها.

UML Class Diagram لنظام UniAccounts.


UniAccounts – UML Class Diagram (Work 1)
«Singleton»
StudentRepository
- instance : StudentRepository
- accounts : List<StudentAccount>
+ getInstance() : StudentRepository
+ add(account : StudentAccount) : void
+ findById(id : String) : StudentAccount
+ getAll() : List<StudentAccount>
«Singleton»
AuditLogger
- instance : AuditLogger
- events : List<String>
+ getInstance() : AuditLogger
+ log(event : String) : void
+ getLogs() : List<String>
1
0..*
0..*
1
records
«Builder»
StudentAccount.Builder
- account : StudentAccount
+ id(id : String) : Builder
+ fullName(name : String) : Builder
+ email(email : String) : Builder
+ phone(phone : String) : Builder
+ level(level : String) : Builder
+ major(major : String) : Builder
+ scholarship(s : boolean) : Builder
+ courses(courses : List<String>) : Builder
+ build() : StudentAccount
builds
StudentAccount
- id : String
- fullName : String
- email : String
- phone : String
- level : String
- major : String
- scholarship : boolean
- courses : List<String>
+ copyWithNewIdentity() : StudentAccount
+ toString() : String
clones
«Prototype»
StudentAccountPrototype
+ cloneWithNewIdentity
(newId : String, newName : String,
newEmail : String) : StudentAccount
creates & manages
uses
uses
Main
+ main(args : String[]) : void

تطبيق أنماط التصميم

2. Builder Pattern

أين استُعمل؟

استُعمل هذا النمط داخل الصنف StudentAccount من خلال الصنف الداخلي StudentAccount.Builder، الذي يسمح بتحديد معلومات الطالب تدريجيًا، مثل المعرف والاسم والبريد الإلكتروني ورقم الهاتف والمستوى الدراسي والتخصص، ثم إنشاء الحساب باستعمال الدالة build().

ما المشكلة التي حلها؟

يسمح هذا النمط بإنشاء حسابات الطلبة بطريقة منظمة ومرنة، دون الحاجة إلى تمرير جميع الخصائص في Constructor واحد، خاصة عندما تكون هناك معلومات اختيارية.

ماذا كان سيحدث لو لم نستعمله؟

كان إنشاء الحسابات سيصبح أقل تنظيمًا، وقد نحتاج إلى عدة Constructors مختلفة حسب المعلومات المقدمة، مما يجعل الكود أكثر تعقيدًا وصعوبة في الصيانة.

3. Singleton Pattern

أين استُعمل؟

تم تطبيق هذا النمط في صنفين:

- StudentRepository: لضمان استعمال نسخة واحدة من مستودع حسابات الطلبة.
- AuditLogger: لضمان استعمال نسخة واحدة من سجل العمليات.

ويتم الوصول إلى النسخة باستعمال الدالة getInstance().

ما المشكلة التي حلها؟

يضمن هذا النمط أن جميع أجزاء البرنامج تستعمل نفس المستودع ونفس سجل العمليات، مما يساعد على تجنب إنشاء نسخ متعددة ومستقلة من هذه المكونات.

ماذا كان سيحدث لو لم نستعمله؟

كان من الممكن إنشاء عدة مستودعات مختلفة، بحيث يحتوي كل واحد منها على بيانات مستقلة، كما يمكن أن تتوز ع سجلات العمليات على عدة نسخ، مما يؤدي إلى صعوبة تتبع البيانات والعمليات.

4. Prototype Pattern

أين استُعمل؟

استُعمل هذا النمط في الصنف StudentAccount من خلال الدالة copyWithNewIdentity()، التي تسمح بإنشاء حساب جديد انطلاقًا من حساب موجود، مع تغيير المعرف والاسم والبريد الإلكتروني والاحتفاظ بباقي المعلومات.

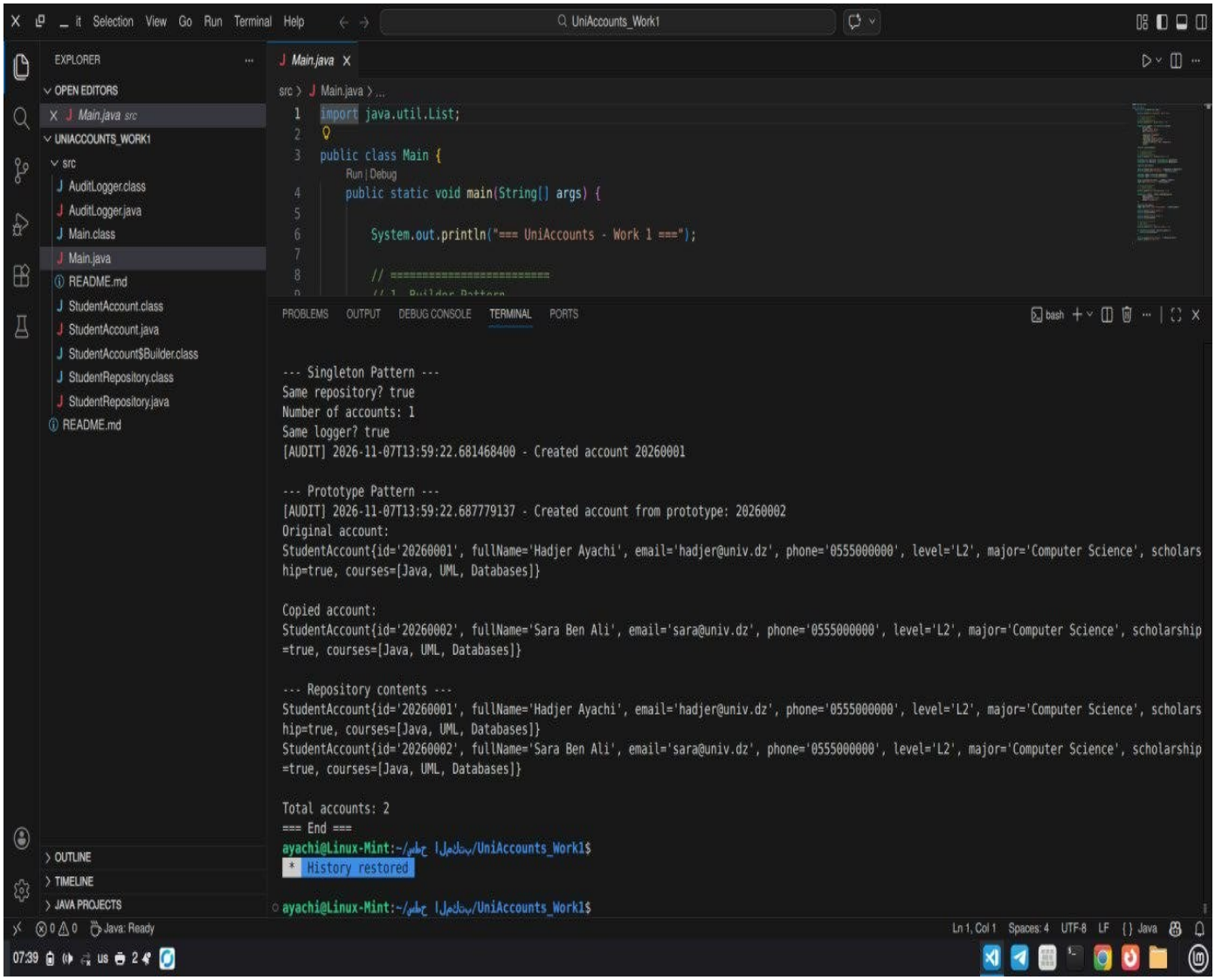
ما المشكلة التي حلها؟

يسمح هذا النمط بإعادة استعمال المعلومات الموجودة في حساب سابق لإنشاء حساب جديد مشابه له، دون الحاجة إلى إعادة إدخال جميع الخصائص من البداية.

ماذا كان سيحدث لو لم نستعمله؟

كنا سنضطر إلى إنشاء حساب جديد وإعادة تحديد جميع خصائصه يدويًا، حتى عندما تكون معظم المعلومات مطابقة لحساب موجود، مما يزيد من تكرار الكود والجهد المبذول.

نتائج التشغيل



تم تنفيذ المشروع باستعمال لغة Java داخل Visual Studio Code، وتم اختبار الأنماط الثلاثة للتحقق من صحة عملها.

5.2. التحقق من Singleton

أظهرت نتيجة التنفيذ:

Same repository? true
Number of accounts: 1
Same logger? true

تؤكد هذه النتيجة أن البرنامج يستعمل نفس النسخة من StudentRepository ونفس النسخة من

.AuditLogger

5.3. التحقق من Prototype

أظهرت نتيجة التنفيذ إنشاء حساب جديد انطلاقًا من الحساب الأصلي، مع تغيير هوية الطالب والاحتفاظ بباقي المعلومات.

الحساب الأصلي:

ID: 20260001
Name: Hadjer Ayachi
Email: hadjer@univ.dz

الحساب المنسوخ:

ID: 20260002
Name: Sara Ben Ali
Email: sara@univ.dz

5.4. النتيجة النهائية

Total accounts: 2
=== End ===

توضح هذه النتيجة نجاح إضافة الحسابين إلى المستودع.
